

# Project Proposal: SENA Network - Layer 2 Modular Infrastructure

Document Version: 1.1.0

## 1. Introduction

This document outlines the proposal for the SENA Network, a Layer 2 (L2) modular infrastructure designed to bridge the gap between traditional Web2 user experiences and the decentralized capabilities of blockchain technology. SENA aims to provide a high-performance, secure, and user-friendly environment for consumer applications by abstracting away the complexities typically associated with Web3 development.

## 2. Problem Statement

The widespread adoption of blockchain technology for consumer applications is hindered by several critical challenges related to user experience and developer friction:

- **Complex Account Management:** Users are often required to manage cryptographic seed phrases, which are difficult to secure and recover, leading to high abandonment rates.
- **Gas Fee Volatility and Management:** The necessity for users to acquire and manage volatile native gas tokens (e.g., ETH, APT) before interacting with applications creates a significant barrier to entry.
- **Impersonal Identifiers:** Transactions rely on long, alphanumeric wallet addresses, lacking the human-readable identifiers common in Web2 platforms.

These issues force application developers to implement complex workarounds or rely on third-party account abstraction solutions, increasing development costs and time-to-market.

A fourth problem sits underneath these three. The usual way to make a chain feel like a Web2 product is to put a trusted operator in the middle — and that quietly returns users to the custodial model they were supposed to be leaving. An L2 that smooths away every rough edge but requires users to trust its sequencer has solved a usability problem by reintroducing a trust problem. SENA treats the elimination of that trust as a product requirement of equal standing with the other three.

### 3. Proposed Solution: SENA Network L2

SENA Network proposes a novel L2 solution that natively embeds identity mappings, Web2 authentication structures, and multi-asset stablecoin gas rails directly into its protocol. By shifting these functionalities from the application layer to the network layer, SENA offers a frictionless Web2-like user experience out-of-the-box, eliminating the need for custom smart-contract relay infrastructures.

SENA is built using the **Sovereign SDK** in **Rust**, leveraging **RocksDB** for high-performance data storage. It anchors its security to the **Aptos Layer 1 (L1)** blockchain, inheriting its robust consensus and finality, and enforces the correctness of that anchoring through an interactive **fraud proof** system adjudicated on Aptos L1.

### 4. Key Features

SENA Network will deliver the following core features:

- **Protocol-Enforced Keyless Authentication Engine:** Native processing of OpenID Connect (OIDC) identities, similar to Aptos Keyless accounts, allowing users to interact with the blockchain using their existing Web2 credentials (e.g., Google, Apple) without managing private keys.
- **Social Connect Identity Index:** An embedded system ledger that maps human-readable identifiers to SENA network addresses, facilitating intuitive social and payment interactions while preserving privacy through cryptographic hashing.
- **Multi-Asset Gas Paymaster Engine:** Support for transaction gas payments using an approved whitelist of stablecoins, removing the requirement for users to hold a native utility token for routine operations.
- **Interactive Fraud Proof System:** Every state root SENA commits to Aptos L1 is posted as a bonded claim, open to permissionless challenge for a defined window. A dispute is resolved by an interactive bisection game that narrows the disagreement to a single machine instruction, which Aptos L1 then executes itself to decide the outcome. No trusted party sits anywhere in that path. A single honest verifier is sufficient to prevent a fraudulent state root from ever finalizing.
- **Optimistic L2 Governance Layer:** A high-throughput, low-overhead governance mechanism enabling token holders to participate in network parameter revisions through off-chain aggregation and on-chain validation, with no authority over the dispute system.

### 5. Technical Overview

SENA's architecture is centered around a high-performance, execution-optimized L2 rollup. The core logic is implemented in Rust using the Sovereign SDK, providing a flexible and secure foundation. Persistent state is managed efficiently using RocksDB, optimized for rapid read/write operations and Merkle tree updates via

NOMT (Nearly-Optimal Merkle Trie).

Security is paramount, with SENA inheriting the robust guarantees of Aptos L1 through periodic cryptographic commitments of its state root. This ensures that even in the event of an L2 server compromise, user funds are secured on the Aptos Mainnet. The Keyless JWT Validation Architecture further enhances security by offloading computationally intensive Zero-Knowledge Proof (ZKP) generation to a decentralized prover network, with lightweight on-chain verification performed by the L2 node.

Committing a state root makes SENA's history immutable; it does not by itself make that history *correct*. A compromised sequencer could commit a state root that no honest execution of the published transactions would produce, and users would withdraw against that false record just as readily as a true one. SENA closes this gap with four mechanisms that operate together. State roots are posted as **bonded assertions** that finalize only after a **challenge window**. The transaction data behind every assertion is **published**, so that anyone can independently re-execute the chain — data that cannot be retrieved is treated as invalid. Any party may **challenge** an assertion without permission, and a challenge is settled by an **interactive bisection game** that narrows the disagreement to one instruction, which a one-step verifier written in Move executes on Aptos L1.

The state transition function is compiled to a deterministic RISC-V target, which the Sovereign SDK's zkVM integration already requires — so the same reproducible binary serves both the fraud proof path and a future validity-proof path, with one set of execution semantics to audit rather than two.

The honest cost of this design is a withdrawal delay: trust-minimised exit to Aptos L1 takes a challenge window. This is intrinsic to the optimistic model rather than an artefact of SENA's implementation. Third-party fast-withdrawal liquidity absorbs it for ordinary amounts, and the eventual move to validity proofs removes it. The delay applies only to L1 exit; transactions on SENA itself confirm in seconds.

## 6. Project Goals and Objectives

The primary goal of the SENA Network project is to become the leading L2 infrastructure for consumer-facing decentralized applications, offering unparalleled user experience and developer efficiency. Specific objectives include:

- Develop and deploy a stable and performant L2 network built on Sovereign SDK, Rust, and RocksDB.
- Implement the full suite of key features: Keyless Authentication, Social Connect Index, Multi-Asset Gas Paymaster, Fraud Proofs, and Optimistic Governance.
- Ensure the "Aptos-Level Secure" model, providing robust security guarantees through L1 cryptographic commitments, ZKP-based authentication, and fraud proofs that make those commitments enforceable by

any party.

- Establish a genuinely independent verifier community, so that the network's security rests on distributed verification rather than on trust in its operator.
- Foster a vibrant ecosystem of developers and applications building on SENA.

## 7. Stakeholders

Key stakeholders for the SENA Network project include:

- **End-Users:** Individuals who will interact with applications built on SENA, benefiting from a seamless Web2-like experience.
- **Application Developers:** Engineers and teams building decentralized applications on the SENA platform.
- **Node Operators:** Entities running SENA L2 sequencers and contributing to network operation.
- **Verifiers:** Independent parties running verifier nodes who police the correctness of committed state and challenge invalid assertions. The security model depends on at least one of them being honest and active.
- **Aptos Foundation/Community:** The broader Aptos ecosystem, providing the underlying L1 security and infrastructure.
- **Investors/Founders:** Individuals or organizations providing financial and strategic support for the project.

## 8. Timeline (High-Level)

- **Phase 1:** Core Protocol Development (Rust, Sovereign SDK, RocksDB Integration), with determinism and a reproducible RISC-V build target established from the outset — these are foundational to fraud proofs and prohibitively expensive to retrofit.
- **Phase 2:** Feature Implementation (Keyless Auth, Social Connect, Gas Paymaster, Governance)
- **Phase 3:** Fraud Proof System (assertion chain and bonding, data availability publication, bisection protocol, one-step verifier in Move, verifier node mode)
- **Phase 4:** Testing & Audits, including adversarial dispute exercises and a dedicated audit scope for the dispute contracts and one-step verifier
- **Phase 5:** Testnet Deployment & Developer Onboarding, with a public verifier programme to establish independent verification before mainnet
- **Phase 6:** Mainnet Launch

## 9. Conclusion

SENA Network represents a significant step forward in making blockchain technology accessible and practical for mainstream consumer applications. By focusing on user experience, developer efficiency, and robust security, SENA is poised to unlock a new era of decentralized innovation — delivering a Web2-grade experience on a settlement layer that asks users to trust no operator, including our own.

## References

[1] Sovereign SDK. Sovereign Labs. Available at: <https://www.sovereign.xyz/> [2] Sovereign SDK Documentation. Sovereign Labs. Available at: <https://docs.sovereign.xyz/> [3] NOMT: Nearly-Optimal Merkle Trie. GitHub. Available at: <https://github.com/aptos-labs/aptos-core/tree/main/storage/aptdb/src/schema/nomt> [4] AptosDB: The Aptos Database. Aptos Docs. Available at: <https://aptos.dev/nodes/aptos-node-references/aptdb/> [5] Merkle Trees. Wikipedia. Available at: [https://en.wikipedia.org/wiki/Merkle\\_tree](https://en.wikipedia.org/wiki/Merkle_tree) [6] Sovereign SDK with NOMT. Sovereign Labs Blog. Available at: <https://www.sovereign.xyz/blog/sovereign-sdk-nomt> [7] RocksDB. GitHub. Available at: <https://github.com/facebook/rocksdb> [8] sov\_db in Sovereign SDK. Sovereign Labs GitHub. Available at: <https://github.com/Sovereign-Labs/sovereign-sdk/tree/main/crates/sov-db> [9] Rust RocksDB Bindings. GitHub. Available at: <https://github.com/rust-rocksdb/rust-rocksdb> [10] Aptos Keyless Accounts. Aptos Docs. Available at: <https://aptos.dev/concepts/accounts/#keyless-accounts> [11] OIDC and ZKP in Aptos. Aptos Labs Blog. Available at: <https://aptoslabs.com/blog/aptos-keyless-accounts-oidc-zkp> [12] Aptos Keyless Account Derivation. Aptos Docs. Available at: <https://aptos.dev/guides/keyless-accounts/> [13] Zero-Knowledge Proofs. Wikipedia. Available at: [https://en.wikipedia.org/wiki/Zero-knowledge\\_proof](https://en.wikipedia.org/wiki/Zero-knowledge_proof) [14] ZKP in Blockchain. CoinDesk. Available at: <https://www.coindesk.com/learn/what-are-zero-knowledge-proofs-zkps/> [15] Decentralized Prover Networks. ZKProof.org. Available at: <https://zkproof.org/> [16] RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. RISC-V International. Available at: <https://riscv.org/technical/specifications/>